

第 4 章

色彩空间类型转换

RGB 图像是一种比较常见的色彩空间类型，除此以外还有一些其他的色彩空间，比较常见的包括 GRAY 色彩空间（灰度图像）、XYZ 色彩空间、YCrCb 色彩空间、HSV 色彩空间、HLS 色彩空间、CIEL*a*b*色彩空间、CIEL*u*v*色彩空间、Bayer 色彩空间等。每个色彩空间都有自己擅长的处理问题的领域，因此，为了更方便地处理某个具体问题，就要用到色彩空间类型转换。

色彩空间类型转换是指，将图像从一个色彩空间转换到另外一个色彩空间。例如，在使用 OpenCV 处理图像时，可能会在 RGB 色彩空间和 HSV 色彩空间之间进行转换。在进行图像的特征提取、距离计算时，往往先将图像从 RGB 色彩空间处理为灰度色彩空间。在一些应用中，可能需要将色彩空间的图像转换为二值图像。

色彩空间也称为颜色空间、彩色空间、颜色模型、彩色系统、彩色模型、色彩模型等。

4.1 色彩空间基础

比较常见的色彩空间包括 GRAY 色彩空间、XYZ 色彩空间、YCrCb 色彩空间、HSV 色彩空间、HLS 色彩空间、CIEL*a*b*色彩空间、CIEL*u*v*色彩空间、Bayer 色彩空间等，下面将依次介绍。

4.1.1 GRAY色彩空间

GRAY（灰度图像）通常指 8 位灰度图，其具有 256 个灰度级，像素值的范围是[0,255]。

当图像由 RGB 色彩空间转换为 GRAY 色彩空间时，其处理方式如下：

$$\text{Gray} = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

上述是标准的转换方式，也是 OpenCV 中使用的转换方式。有时，也可以采用简化形式完成转换：

$$\text{Gray} = \frac{R + G + B}{3}$$

当图像由 GRAY 色彩空间转换为 RGB 色彩空间时，最终所有通道的值都将是相同的，其处理方式如下：

$$R = \text{Gray}$$

$$G = \text{Gray}$$



$$B = \text{Gray}$$

4.1.2 XYZ色彩空间

XYZ 色彩空间是由 CIE（International Commission on Illumination）定义的，是一种更便于计算的色彩空间，它可以与 RGB 色彩空间相互转换。

将 RGB 色彩空间转换为 XYZ 色彩空间，其转换形式为：

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} 0.412453 & 0.357580 & 0.180423 \\ 0.212671 & 0.715160 & 0.072169 \\ 0.019334 & 0.119193 & 0.950227 \end{bmatrix} \cdot \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

将 XYZ 色彩空间转换为 RGB 色彩空间，其转换形式为：

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 3.240479 & -1.53715 & -0.498535 \\ -0.969256 & 1.875991 & 0.041556 \\ 0.055648 & -0.204043 & 1.057311 \end{bmatrix} \cdot \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

4.1.3 YCrCb色彩空间

人眼视觉系统（HVS, Human Visual System）对颜色的敏感度要低于对亮度的敏感度。在传统的 RGB 色彩空间内，RGB 三原色具有相同的重要性，但是忽略了亮度信息。

在 YCrCb 色彩空间中，Y 代表光源的亮度，色度信息保存在 Cr 和 Cb 中，其中，Cr 表示红色分量信息，Cb 表示蓝色分量信息。

亮度给出了颜色亮或暗的程度信息，该信息可以通过照明中强度成分的加权和来计算。在 RGB 光源中，绿色分量的影响最大，蓝色分量的影响最小。

从 RGB 色彩空间到 YCrCb 色彩空间的转换公式为：

$$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

$$\text{Cr} = (R - Y) \times 0.713 + \text{delta}$$

$$\text{Cb} = (B - Y) \times 0.564 + \text{delta}$$

式中 delta 的值为：

$$\text{delta} = \begin{cases} 128, & 8 \text{ 位图像} \\ 32768, & 16 \text{ 位图像} \\ 0.5, & \text{单精度图像} \end{cases}$$

从 YCrCb 色彩空间到 RGB 色彩空间的转换公式为：

$$R = Y + 1.403 \cdot (\text{Cr} - \text{delta})$$

$$G = Y - 0.714 \cdot (\text{Cr} - \text{delta}) - 0.344 \cdot (\text{Cb} - \text{delta})$$

$$B = Y + 1.773 \cdot (\text{Cb} - \text{delta})$$

式中，delta 的值与上面公式中的 delta 值相同。

4.1.4 HSV色彩空间

RGB 是从硬件的角度提出的颜色模型，在与入眼匹配的过程中可能存在一定的差异，HSV 色彩空间是一种面向视觉感知的颜色模型。HSV 色彩空间从心理学和视觉的角度出发，指出人眼的色彩知觉主要包含三要素：色调（Hue，也称为色相）、饱和度（Saturation）、亮度（Value），色调指光的颜色，饱和度是指色彩的深浅程度，亮度指人眼感受到的光的明暗程度。

- 色调：色调与混合光谱中的主要光波长相关，例如“赤橙黄绿青蓝紫”分别表示不同的色调。如果从波长的角度考虑，不同波长的光表现为不同的颜色，实际上它们体现的是色调的差异。
- 饱和度：指相对纯净度，或一种颜色混合白光的数量。纯谱色是全饱和的，像深红色（红加白）和淡紫色（紫加白）这样的彩色是欠饱和的，饱和度与所加白光的数量成反比。
- 亮度：反映的是人眼感受到的光的明暗程度，该指标与物体的反射度有关。对于色彩来讲，如果在其中掺入的白色越多，则其亮度越高；如果在其中掺入的黑色越多，则其亮度越低。

在具体实现上，我们将物理空间的颜色分布在圆周上，不同的角度代表不同的颜色。因此，通过调整色调值就能选取不同的颜色，色调的取值区间为[0, 360]。色调取不同值时，所代表的颜色如表 4-1 所示，两个角度之间的角度对应两个颜色之间的过渡色。

表 4-1 色调环

色调值（度）	颜色
0	红色
60	黄色
120	绿色
180	青色
240	蓝色
300	品红色

饱和度为一比例值，范围是[0, 1]，具体为所选颜色的纯度值和该颜色最大纯度值之间的比值。饱和度的值为 0 时，只有灰度。亮度表示色彩的明亮程度，取值范围也是[0, 1]。

在 HSV 色彩模型中，取色变得更加直观。例如，取值“色调=0，饱和度=1，亮度=1”，则当前色彩为深红色，而且颜色较亮；取值“色调=120，饱和度=0.3，亮度=0.4”，则当前色彩为浅绿色，而且颜色较暗。

在从 RGB 色彩空间转换到 HSV 色彩空间之前，需要先将 RGB 色彩空间的值转换到[0, 1]之间，然后再进行处理。具体处理方法为：

$$V = \max(R, G, B)$$

$$S = \begin{cases} \frac{V - \min(R, G, B)}{V}, & V \neq 0 \\ 0, & \text{其他情况} \end{cases}$$



$$H = \begin{cases} \frac{60(G - B)}{V - \min(R, G, B)}, & V = R \\ 120 + \frac{60(B - R)}{V - \min(R, G, B)}, & V = G \\ 240 + \frac{60(R - G)}{V - \min(R, G, B)}, & V = B \end{cases}$$

计算结果可能存在 $H < 0$ 的情况，如果出现这种情况，则需要对 H 进行进一步计算，如下。

$$H = \begin{cases} H + 360, & H < 0 \\ H, & \text{其他情况} \end{cases}$$

由上述公式计算可知：

$$S \in [0, 1]$$

$$V \in [0, 1]$$

$$H \in [0, 360]$$

当然，也可以通过公式完成从 HSV 色彩空间到 RGB 色彩空间的转换。在 OpenCV 的官方文档中有完整的转换公式，这里不再赘述。

当然，所有这些转换都被封装在 OpenCV 的 `cv2.cvtColor()` 函数内。通常情况下，我们都是直接调用该函数来完成色彩空间转换的，而不用考虑函数的内部实现细节。

本章的后续小节还会更细致地讨论 HSV 色彩空间的使用情况。

4.1.5 HLS 色彩空间

HLS 色彩空间包含的三要素是色调 H (Hue)、光亮度/明度 L (Lightness)、饱和度 S (Saturation)。

与 HSV 色彩空间类似，只是 HLS 色彩空间用“光亮度/明度 L (lightness)”替换了“亮度 (Value)”。

- 色调：表示人眼所能感知的颜色，在 HLS 模型中，所有的颜色分布在一个平面的色调环上，整个色调环为 360 度的圆心角，不同的角度代表不同的颜色，如表 4-1 所示。
- 光亮度/明度：用来控制色彩的明暗变化，它的取值范围也是 $[0, 1]$ 。我们通过光亮度/明度的大小来衡量有多少光线从物体表面反射出来。光亮度/明度对于眼睛感知颜色很重要，因为当一个具有色彩的物体处于光线太强或者光线太暗的地方时，眼睛是无法准确感知物体颜色的。
- 饱和度：使用 $[0, 1]$ 的值描述相同色调、相同光亮度/明度下的色彩纯度变化。饱和度的值越大，表示颜色的纯度越高，颜色越鲜艳；反之，饱和度的值越小，色彩的纯度越低，颜色越暗沉。通常用该属性表示颜色的深浅，比如深绿色、浅绿色。

4.1.6 CIE L*a*b* 色彩空间

CIE L*a*b* 色彩空间是均匀色彩空间模型，它是面向视觉感知的颜色模型。从视觉感知均

匀的角度来讲,人所感知到的两种颜色的区别程度,应该与这两种颜色在色彩空间中的距离成正比。在某个色彩空间中,如果人所观察到的两种颜色的区别程度,与这两种颜色在该色彩空间中对应的点之间的欧式距离成正比,则称该色彩空间为均匀色彩空间。

CIEL*a*b*色彩空间中的 L*分量用于表示像素的亮度,取值范围是[0,100],表示从纯黑到纯白;a*分量表示从红色到绿色的范围,取值范围是[-127,127];b*分量表示从黄色到蓝色的范围,取值范围是[-127,127]。

在从 RGB 色彩空间转换到 CIEL*a*b*色彩空间之前,需要先将 RGB 色彩空间的值转换到 [0, 1]之间,然后再进行处理。

由于 CIEL*a*b*色彩空间是在 CIE 的 XYZ 色彩空间的基础上发展起来的,在具体处理时,需要先将 RGB 转换为 XYZ 色彩空间,再将其转换到 CIEL*a*b*色彩空间。具体实现方法为:

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} 0.412453 & 0.357580 & 0.180423 \\ 0.212671 & 0.715160 & 0.072169 \\ 0.019334 & 0.119193 & 0.950227 \end{bmatrix} \cdot \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

$$X = \frac{X}{X_n}, \quad X_n = 0.950456$$

$$Z = \frac{Z}{Z_n}, \quad Z_n = 1.088754$$

$$L = \begin{cases} 116 \cdot Y^{\frac{1}{3}} - 16, & Y > 0.008856 \\ 903.3 \cdot Y, & \text{其他情况} \end{cases}$$

$$a = 500 \cdot (f(X) - f(Y)) + \text{delta}$$

$$b = 200 \cdot (f(Y) - f(Z)) + \text{delta}$$

式中:

$$f(t) = \begin{cases} t^{\frac{1}{3}}, & t > 0.008856 \\ 7.787t + \frac{16}{116}, & \text{其他情况} \end{cases}$$

$$\text{delta} = \begin{cases} 128, & \text{8 位图像} \\ 0, & \text{单精度图像} \end{cases}$$

所得结果中各个值的取值范围为:

$$L \in [0, 100]$$

$$a \in [-127, 127]$$

$$b \in [-127, 127]$$

4.1.7 CIEL*u*v*色彩空间

CIEL*u*v*色彩空间同 CIEL*a*b*色彩空间一样,都是均匀的颜色模型。CIEL*u*v*色彩空间与设备无关,适用于显示器显示和根据加色原理进行组合的场合,该模型中比较强调对红色的表示,即对红色的变化比较敏感,但对蓝色的变化不太敏感。

下面的公式给出了从 RGB 色彩空间到 CIEL*u*v*色彩空间的转换公式。

从 RGB 色彩空间到 XYZ 色彩空间的转换:



$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} 0.412453 & 0.357580 & 0.180423 \\ 0.212671 & 0.715160 & 0.072169 \\ 0.019334 & 0.119193 & 0.950227 \end{bmatrix} \cdot \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

从 XYZ 色彩空间到 CIE L*u*v*色彩空间的转换:

$$L = \begin{cases} 116 \cdot Y^{\frac{1}{3}} - 16, & Y > 0.008856 \\ 903.3 \cdot Y, & \text{其他情况} \end{cases}$$
$$u' = \frac{4X}{X + 15Y + 3Z}$$
$$v' = \frac{9Y}{X + 15Y + 3Z}$$
$$u = 13 \cdot L \cdot (u' - u_n), \quad u_n = 0.19793943$$
$$v = 13 \cdot L \cdot (v' - v_n), \quad v_n = 0.46831096$$

所得结果中各个值的取值范围分别为:

$$L \in [0,100]$$
$$u \in [-134,220]$$
$$v \in [-140,122]$$

4.1.8 Bayer色彩空间

Bayer 色彩空间（Bayer 模型）被广泛地应用在 CCD 和 CMOS 相机中。它能够从如图 4-1 所示的单平面 R、G、B 交错表内获取彩色图像。

R	G	R	G	R
G	B	G	B	G
R	G	R	G	R
G	B	G	B	G
R	G	R	G	R

图 4-1 Bayer 色彩空间交错表

输出的 RGB 图像的像素点值，是根据当前点的 1 个、2 个或 4 个邻域像素点的相同颜色的像素值获得的。上述模式能够通过移动一个左边的像素或者上方的像素来完成修改。在函数 cv2.cvtColor() 的色彩空间转换参数中，通常使用两个特定的参数 x 和 y 来表示特定的模式。该模式组成通过图 4-1 第二行中的第 2 列与第 3 列的值来指定。图 4-1 就是典型的“BG”模式。

常见的模式还有很多，例如 4.2 节即将展示的表 4-2 中的 cv2.COLOR_BayerBG2BGR、cv2.COLOR_BayerGB2BGR、cv2.COLOR_BayerRG2BGR、cv2.COLOR_BayerGR2BGR、cv2.COLOR_BayerBG2RGB、cv2.COLOR_BayerGB2RGB、cv2.COLOR_BayerRG2RGB、cv2.COLOR_BayerGR2RGB 等。

4.2 类型转换函数

在 OpenCV 内，我们使用 cv2.cvtColor() 函数实现色彩空间的变换。该函数能够实现多个色彩空间之间的转换。其语法格式为：

```
dst = cv2.cvtColor( src, code [, dstCn] )
```

式中：

- **dst** 表示输出图像，与原始输入图像具有同样的数据类型和深度。
- **src** 表示原始输入图像。可以是 8 位无符号图像、16 位无符号图像，或者单精度浮点数等。
- **code** 是色彩空间转换码，表 4-2 展示了其枚举值。
- **dstCn** 是目标图像的通道数。如果参数为默认的 0，则通道数自动通过原始输入图像和 **code** 得到。

表 4-2 枚举值

值	备注
cv2.COLOR_BGR2BGR	为 BGR 或 RGB 图像添加 alpha 通道（alpha 是透明属性）
cv2.COLOR_RGB2BGR	
cv2.COLOR_BGR2BGRA	从 BGR 或 RGB 通道内删除 alpha 通道
cv2.COLOR_RGB2BGRA	
cv2.COLOR_BGR2RGB	在 BGR 和 RGB 色彩空间之间转换（包含或者不包含 alpha 通道）
cv2.COLOR_RGB2BGR	
cv2.COLOR_BGR2RGBA	
cv2.COLOR_RGBA2BGR	
cv2.COLOR_RGB2BGR	
cv2.COLOR_BGRA2RGB	
cv2.COLOR_RGBA2BGR	
cv2.COLOR_RGBA2BGRA	
cv2.COLOR_BGR2GRAY	在 RGB/BGR 和灰度图像之间转换
cv2.COLOR_RGB2GRAY	
cv2.COLOR_GRAY2BGR	
cv2.COLOR_GRAY2RGB	
cv2.COLOR_GRAY2BGRA	
cv2.COLOR_GRAY2RGBA	
cv2.COLOR_BGRA2GRAY	
cv2.COLOR_RGBA2GRAY	
cv2.COLOR_BGR2BGR565	在 RGB/BGR 和 BGR565（16 位图像）之间转换
cv2.COLOR_RGB2BGR565	
cv2.COLOR_BGR5652BGR	
cv2.COLOR_BGR5652RGB	
cv2.COLOR_BGRA2BGR565	
cv2.COLOR_RGBA2BGR565	
cv2.COLOR_BGR5652BGRA	
cv2.COLOR_BGR5652RGBA	
cv2.COLOR_GRAY2BGR565	在灰度图像和 BGR565（16 位图像）之间转换
cv2.COLOR_BGR5652GRAY	
cv2.COLOR_BGR2BGR555	在 RGB/BGR 和 BGR555（16 位图像）之间转换
cv2.COLOR_RGB2BGR555	
cv2.COLOR_BGR5552BGR	
cv2.COLOR_BGR5552RGB	

续表

值	备注
cv2.COLOR_BGRA2BGR555	在 RGB/BGR 和 BGR555（16 位图像）之间转换
cv2.COLOR_RGBA2BGR555	
cv2.COLOR_BGR5552BGRA	
cv2.COLOR_BGR5552RGBA	
cv2.COLOR_GRAY2BGR555	在灰度图像和 BGR555（16 位图像）之间转换
cv2.COLOR_BGR5552GRAY	
cv2.COLOR_BGR2XYZ	在 RGB/BGR 和 CIE XYZ 之间转换
cv2.COLOR_RGB2XYZ	
cv2.COLOR_XYZ2BGR	
cv2.COLOR_XYZ2RGB	
cv2.COLOR_BGR2YCrCb	在 RGB/BGR 和 luma-chroma (aka YCC) 之间转换
cv2.COLOR_RGB2YCrCb	
cv2.COLOR_YCrCb2BGR	
cv2.COLOR_YCrCb2RGB	
cv2.COLOR_BGR2HSV	将 RGB/BGR 转换为 HSV（Hue Saturation Value）
cv2.COLOR_RGB2HSV	
cv2.COLOR_BGR2Lab	将 RGB/BGR 转换为 CIE Lab
cv2.COLOR_RGB2Lab	
cv2.COLOR_BGR2Luv	将 RGB/BGR 转换为 CIE Luv
cv2.COLOR_RGB2Luv	
cv2.COLOR_BGR2HLS	将 RGB/BGR 转换为 HLS（Hue Lightness Saturation）
cv2.COLOR_RGB2HLS	
cv2.COLOR_HSV2BGR	转换回 RGB/BGR
cv2.COLOR_HSV2RGB	
cv2.COLOR_Lab2BGR	
cv2.COLOR_Lab2RGB	
cv2.COLOR_Luv2BGR	
cv2.COLOR_Luv2RGB	
cv2.COLOR_HLS2BGR	
cv2.COLOR_HLS2RGB	
cv2.COLOR_BGR2HSV_FULL	
cv2.COLOR_RGB2HSV_FULL	
cv2.COLOR_BGR2HLS_FULL	
cv2.COLOR_RGB2HLS_FULL	
cv2.COLOR_HSV2BGR_FULL	
cv2.COLOR_HSV2RGB_FULL	
cv2.COLOR_HLS2BGR_FULL	
cv2.COLOR_HLS2RGB_FULL	
cv2.COLOR_LBGR2Lab	
cv2.COLOR_LRGB2Lab	
cv2.COLOR_LBGR2Luv	
cv2.COLOR_LRGB2Luv	



续表

值	备注
cv2.COLOR_Lab2LBGR	转换回 RGB/BGR
cv2.COLOR_Lab2LRGB	
cv2.COLOR_Luv2LBGR	
cv2.COLOR_Luv2LRGB	
cv2.COLOR_BGR2YUV	在 RGB/BGR 和 YUV 之间转换
cv2.COLOR_RGB2YUV	
cv2.COLOR_YUV2BGR	
cv2.COLOR_YUV2RGB	
cv2.COLOR_YUV2RGB_NV12	将 YUV 4:2:0 族 (family) 转换为 RGB
cv2.COLOR_YUV2BGR_NV12	
cv2.COLOR_YUV2RGB_NV21	
cv2.COLOR_YUV2BGR_NV21	
cv2.COLOR_YUV420sp2RGB	
cv2.COLOR_YUV420sp2BGR	
cv2.COLOR_YUV2RGBA_NV12	
cv2.COLOR_YUV2BGRA_NV12	
cv2.COLOR_YUV2RGBA_NV21	
cv2.COLOR_YUV2BGRA_NV21	
cv2.COLOR_YUV420sp2RGBA	
cv2.COLOR_YUV420sp2BGRA	
cv2.COLOR_YUV2RGB_YV12	
cv2.COLOR_YUV2BGR_YV12	
cv2.COLOR_YUV2RGB_IYUV	
cv2.COLOR_YUV2BGR_IYUV	
cv2.COLOR_YUV2RGB_I420	
cv2.COLOR_YUV2BGR_I420	
cv2.COLOR_YUV420p2RGB	
cv2.COLOR_YUV420p2BGR	
cv2.COLOR_YUV2RGBA_YV12	
cv2.COLOR_YUV2BGRA_YV12	
cv2.COLOR_YUV2RGBA_IYUV	
cv2.COLOR_YUV2BGRA_IYUV	
cv2.COLOR_YUV2RGBA_I420	
cv2.COLOR_YUV2BGRA_I420	
cv2.COLOR_YUV420p2RGBA	
cv2.COLOR_YUV420p2BGRA	
cv2.COLOR_YUV2GRAY_420	
cv2.COLOR_YUV2GRAY_NV21	
cv2.COLOR_YUV2GRAY_NV12	
cv2.COLOR_YUV2GRAY_YV12	
cv2.COLOR_YUV2GRAY_IYUV	
cv2.COLOR_YUV2GRAY_I420	



续表

值	备注
cv2.COLOR_YUV420sp2GRAY	将 YUV 4:2:2 族转换为 RGB
cv2.COLOR_YUV420p2GRAY	
cv2.COLOR_YUV2RGB_UYVY	
cv2.COLOR_YUV2BGR_UYVY	
cv2.COLOR_YUV2RGB_Y422	
cv2.COLOR_YUV2BGR_Y422	
cv2.COLOR_YUV2RGB_UYNV	
cv2.COLOR_YUV2BGR_UYNV	
cv2.COLOR_YUV2RGBA_UYVY	
cv2.COLOR_YUV2BGRA_UYVY	
cv2.COLOR_YUV2RGBA_Y422	
cv2.COLOR_YUV2BGRA_Y422	
cv2.COLOR_YUV2RGBA_UYNV	
cv2.COLOR_YUV2BGRA_UYNV	
cv2.COLOR_YUV2RGB_YUY2	
cv2.COLOR_YUV2BGR_YUY2	
cv2.COLOR_YUV2RGB_YVYU	
cv2.COLOR_YUV2BGR_YVYU	
cv2.COLOR_YUV2RGB_YUYV	
cv2.COLOR_YUV2BGR_YUYV	
cv2.COLOR_YUV2RGB_YUNV	
cv2.COLOR_YUV2BGR_YUNV	
cv2.COLOR_YUV2RGBA_YUY2	
cv2.COLOR_YUV2BGRA_YUY2	
cv2.COLOR_YUV2RGBA_YVYU	
cv2.COLOR_YUV2BGRA_YVYU	
cv2.COLOR_YUV2RGBA_YUYV	
cv2.COLOR_YUV2BGRA_YUYV	
cv2.COLOR_YUV2RGBA_YUNV	
cv2.COLOR_YUV2BGRA_YUNV	
cv2.COLOR_YUV2GRAY_UYVY	
cv2.COLOR_YUV2GRAY_YUY2	
cv2.COLOR_YUV2GRAY_Y422	
cv2.COLOR_YUV2GRAY_UYNV	
cv2.COLOR_YUV2GRAY_YVYU	
cv2.COLOR_YUV2GRAY_YUYV	
cv2.COLOR_YUV2GRAY_YUNV	
cv2.COLOR_RGBA2mRGBA	Alpha 预乘 (Alpha premultiplication)
cv2.COLOR_mRGBA2RGBA	
cv2.COLOR_RGB2YUV_I420	将 RGB 族转换为 YUV 4:2:0 族
cv2.COLOR_BGR2YUV_I420	
cv2.COLOR_RGB2YUV_IYUV	

续表

值	备注
cv2.COLOR_BGR2YUV_IYUV	将 RGB 族转换为 YUV 4:2:0 族
cv2.COLOR_RGBA2YUV_I420	
cv2.COLOR_BGRA2YUV_I420	
cv2.COLOR_RGBA2YUV_IYUV	
cv2.COLOR_BGRA2YUV_IYUV	
cv2.COLOR_RGB2YUV_YV12	
cv2.COLOR_BGR2YUV_YV12	
cv2.COLOR_RGBA2YUV_YV12	
cv2.COLOR_BGRA2YUV_YV12	
cv2.COLOR_BayerBG2BGR	逆马赛克 (Demosaicing)
cv2.COLOR_BayerGB2BGR	
cv2.COLOR_BayerRG2BGR	
cv2.COLOR_BayerGR2BGR	
cv2.COLOR_BayerBG2RGB	
cv2.COLOR_BayerGB2RGB	
cv2.COLOR_BayerRG2RGB	
cv2.COLOR_BayerGR2RGB	
cv2.COLOR_BayerBG2GRAY	
cv2.COLOR_BayerGB2GRAY	
cv2.COLOR_BayerRG2GRAY	
cv2.COLOR_BayerGR2GRAY	
cv2.COLOR_BayerBG2BGR_VNG	使用可变数量的梯度实现逆马赛克 (Demosaicing using variable number of gradients)
cv2.COLOR_BayerGB2BGR_VNG	
cv2.COLOR_BayerRG2BGR_VNG	
cv2.COLOR_BayerGR2BGR_VNG	
cv2.COLOR_BayerBG2RGB_VNG	
cv2.COLOR_BayerGB2RGB_VNG	
cv2.COLOR_BayerRG2RGB_VNG	
cv2.COLOR_BayerGR2RGB_VNG	
cv2.COLOR_BayerBG2BGR_EA	边缘感知逆马赛克 (Edge-Aware Demosaicing)
cv2.COLOR_BayerGB2BGR_EA	
cv2.COLOR_BayerRG2BGR_EA	
cv2.COLOR_BayerGR2BGR_EA	
cv2.COLOR_BayerBG2RGB_EA	
cv2.COLOR_BayerGB2RGB_EA	
cv2.COLOR_BayerRG2RGB_EA	
cv2.COLOR_BayerGR2RGB_EA	
cv2.COLOR_BayerBG2BGRA	用 alpha 通道实现逆马赛克 (Demosaicing with alpha channel)
cv2.COLOR_BayerGB2BGRA	
cv2.COLOR_BayerRG2BGRA	
cv2.COLOR_BayerGR2BGRA	
cv2.COLOR_BayerBG2RGBA	



续表

值	备注
cv2.COLOR_BayerGB2RGBA	用 alpha 通道实现逆马赛克（Demosaicing with alpha channel）
cv2.COLOR_BayerRG2RGBA	
cv2.COLOR_BayerGR2RGBA	
cv2.COLOR_COLORCVT_MAX	

这里需要注意，BGR 色彩空间与传统的 RGB 色彩空间不同。对于一个标准的 24 位位图，BGR 色彩空间中第 1 个 8 位（第 1 个字节）存储的是蓝色组成信息（Blue component），第 2 个 8 位（第 2 个字节）存储的是绿色组成信息（Green component），第 3 个 8 位（第 3 个字节）存储的是红色组成信息（Red component）。同样，其第 4 个、第 5 个、第 6 个字节分别存储蓝色、绿色、红色组成信息，以此类推。

颜色空间的转换都用到了如下约定：

- 8 位图像值的范围是[0,255]。
- 16 位图像值的范围是[0,65 535]。
- 浮点数图像值的范围是[0.0~1.0]。

对于线性转换来说，这些取值范围是无关紧要的。但是对于非线性转换来说，输入的 RGB 图像必须归一化到其对应的取值范围内，才能获取正确的转换结果。

例如，对于 8 位图，其能够表示的灰度级有 $2^8=256$ 个，也就是说，在 8 位图中，最多能表示 256 个状态，通常是[0,255]之间的值。但是，在很多色彩空间中，值的范围并不恰好在[0,255]范围内，这时，就需要将该值映射到[0,255]内。

例如，在 HSV 或 HLS 色彩空间中，色调值通常在[0,360)范围内，在 8 位图中转换到上述色彩空间后，色调值要除以 2，让其值范围变为[0,180)，以满足存储范围，即让值的分布位于 8 位图能够表示的范围[0,255]内。又例如，在 CIEL*a*b*色彩空间中，a 通道和 b 通道的值范围是[-127,127]，为了使其适应[0,255]的范围，每个值都要加上 127。不过需要注意，由于计算过程存在四舍五入，所以转换过程并不是精准可逆的。

4.3 类型转换实例

本节介绍几种常用的色彩空间转换实例，以帮助大家更好地理解应该如何使用函数 cv2.cvtColor()。

4.3.1 通过数组观察转换效果

本节通过使用函数 cv2.cvtColor()的实例来观察它的色彩空间转换功能。为了方便观察，本节通过一个小尺寸的数组来模拟图像，作为 cv2.cvtColor()函数要处理的对象。

【例 4.1】将 BGR 图像转换为灰度图像。

根据题目要求，设计程序如下：

```
import cv2
import numpy as np
img=np.random.randint(0,256,size=[2,4,3],dtype=np.uint8)
rst=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
print("img=\n",img)
print("rst=\n",rst)
print("像素点(1,0)直接计算得到的值=",
      img[1,0,0]*0.114+img[1,0,1]*0.587+img[1,0,2]*0.299)
print("像素点(1,0)使用公式 cv2.cvtColor()转换值=",rst[1,0])
```

在本例中，通过函数 `cv2.cvtColor()` 对图像 `img` 进行色彩空间的转换，并使用公式计算的方式对其中的像素点进行计算。最后，分别打印该点的函数转换结果和公式计算结果。

运行程序，结果如下所示：

```
img=
[[[166  97 102]
  [ 73  31  51]
  [221  94 158]
  [143 101 172]]

 [[ 96 140  92]
  [ 19 156  34]
  [ 16 113 202]
  [116 236 238]]]
rst=
[[106  42 128 127]
 [121 104 129 223]]
像素点(1,0)直接计算得到的值= 120.63199999999999
像素点(1,0)使用公式 cv2.cvtColor()转换值= 121
```

在 OpenCV 中，灰度图像是按照行列直接存储的。而 BGR 模式的图像会依次将它的 B 通道、G 通道、R 通道中的像素点，以行为单位按照顺序存储在 `ndarray` 的列中。例如，有大小为 `R` 行×`C` 列的 BGR 图像，其存储方式如图 4-2 所示。

当图像由 RGB 色彩空间转换到 GRAY 色彩空间时，其处理方式如下：

$$\text{Gray} = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

这里以 `rst[1,0]` 为例，看看它是如何通过计算得到的。像素 `rst[1,0]` 表示目标灰度图像内的第 1 行第 0 列上的像素点。它需要通过对原始图像 BGR 图像内第 1 行第 0 列上的 B 通道像素点、G 通道像素点、R 通道像素点的计算得到。

在本例中，各个像素点的像素值如下：

- 原始图像 BGR 图像内第 1 行第 0 列上的 B 通道像素点的值为 `img[1,0,0]=96`。
- 原始图像 BGR 图像内第 1 行第 0 列上的 G 通道像素点的值为 `img[1,0,1]=140`。
- 原始图像 BGR 图像内第 1 行第 0 列上的 R 通道像素点的值为 `img[1,0,2]=92`。



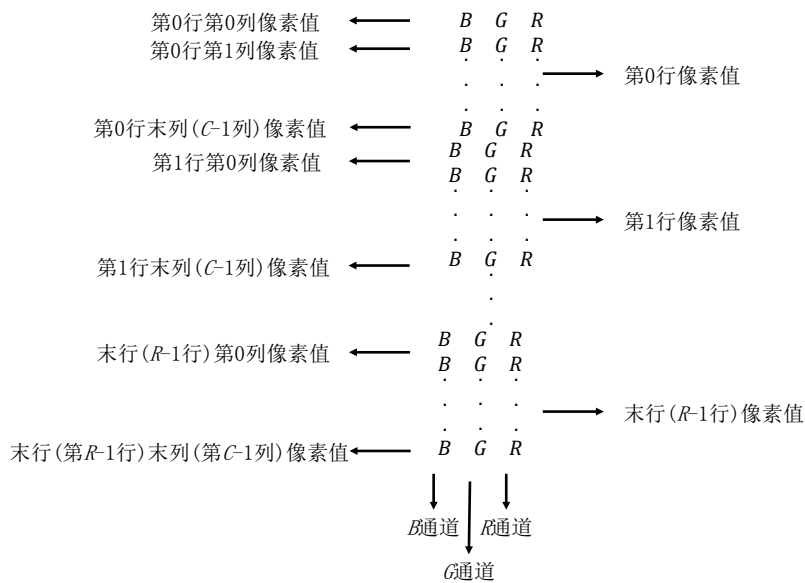


图 4-2 BGR 图像的存储方式

转换为灰度像素点时，使用的公式为：

$$\begin{aligned} & \text{img}[1,0,0] \times 0.114 + \text{img}[1,0,1] \times 0.587 + \text{img}[1,0,2] \times 0.299 \\ &= 96 \times 0.114 + 140 \times 0.587 + 92 \times 0.299 \\ &= 120.632 \end{aligned}$$

计算结果为 120.632。目标图像是灰度图像，是 8 位图像，值是位于[0,255]之间的无符号整数。所以，要将上述小数结果进行四舍五入，得到 121，并将它作为目标灰度图像内 rst[1,0] 的像素值。

当然，在一般情况下，我们并不需要关心这么细节的问题，只要能够熟练地使用函数 cv2.cvtColor()就可以了。

【例 4.2】将灰度图像转换为 BGR 图像。

根据题目要求，设计程序如下：

```
import cv2
import numpy as np
img=np.random.randint(0,256,size=[2,4],dtype=np.uint8)
rst=cv2.cvtColor(img,cv2.COLOR_GRAY2BGR)
print("img=\n",img)
print("rst=\n",rst)
```

运行程序，结果如下所示：

```
img=
[[ 58 254 137 123]
 [100 150 7 42]]
rst=
[[[ 58 58 58]
```

```
[254 254 254]
[137 137 137]
[123 123 123]]

[[100 100 100]
 [150 150 150]
 [ 7  7  7]
 [ 42 42 42]]]
```

上述程序进一步印证了，当图像由 GRAY 色彩空间转换到 RGB/BGR 色彩空间时，最终所有通道的值都是相同的。其处理方式如下：

$$R = \text{Gray}$$

$$G = \text{Gray}$$

$$B = \text{Gray}$$

同时，以上例题也进一步印证了 BGR 图像在 OpenCV 内的像素点的存储方式。

【例 4.3】 将图像在 BGR 和 RGB 模式之间相互转换。

根据题目要求，设计程序如下：

```
import cv2
import numpy as np
img=np.random.randint(0,256,size=[2,4,3],dtype=np.uint8)
rgb=cv2.cvtColor(img,cv2.COLOR_BGR2RGB)
bgr=cv2.cvtColor(rgb,cv2.COLOR_RGB2BGR)
print("img=\n",img)
print("rgb=\n",rgb)
print("bgr=\n",bgr)
```

运行程序，结果如下所示：

```
img=
[[[ 12 173 157]
 [216  2 220]
 [ 49 145 157]
 [124 203 44]]

 [[140 196 171]
 [ 90 195 187]
 [158 199 113]
 [213 78 164]]]

rgb=
[[[157 173 12]
 [220  2 216]
 [157 145 49]
 [ 44 203 124]]]
```



```

[[171 196 140]
 [187 195  90]
 [113 199 158]
 [164  78 213]]]
bgr=
[[[ 12 173 157]
 [216  2 220]
 [ 49 145 157]
 [124 203  44]]

 [[140 196 171]
 [ 90 195 187]
 [158 199 113]
 [213  78 164]]]

```

从程序可以看到，在 RGB 和 BGR 模式之间相互转换时，R 通道和 B 通道的位置发生了交换。

4.3.2 图像处理实例

本节将通过具体实例介绍如何使用函数 `cv2.cvtColor()` 来处理图像类型的转换。

【例 4.4】 将图像在 BGR 模式和灰度图像之间相互转换。

根据题目要求，设计程序如下：

```

import cv2
lena=cv2.imread("lenacolor.png")
gray=cv2.cvtColor(lena,cv2.COLOR_BGR2GRAY)
rgb=cv2.cvtColor(gray,cv2.COLOR_GRAY2BGR)
#=====打印 shape=====
print("lena.shape=",lena.shape)
print("gray.shape=",gray.shape)
print("rgb.shape=",rgb.shape)
#=====显示效果=====
cv2.imshow("lena",lena)
cv2.imshow("gray",gray)
cv2.imshow("rgb",rgb)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行程序，会显示各个图像的 shape 属性：

```

lena.shape= (512, 512, 3)
gray.shape= (512, 512)
rgb.shape= (512, 512, 3)

```

通过其 shape 属性，可以看到图像在转换前后的色彩空间变化情况。

同时，程序会分别显示原始彩色图像、灰度图像、RGB 图像。需要注意，在通过“`rgb=cv2.cvtColor(gray,cv2.COLOR_GRAY2BGR)`”得到的 RGB 图像中，B 通道、G 通道、R

通道的值都是一样的，所以其看起来仍是灰度图像。由于本书是黑白（灰度）印刷，无法观察彩色效果，所以，这里省略了运行结果中图像的展示。请大家在 Python 内运行上述程序，观察实际效果。

【例 4.5】将图像从 BGR 模式转换为 RGB 模式。

根据题目要求，设计程序如下：

```
import cv2
lena=cv2.imread("lenacolor.png")
rgb = cv2.cvtColor(lena, cv2.COLOR_BGR2RGB)
cv2.imshow("lena", lena)
cv2.imshow("rgb", rgb)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，会显示如图 4-3 所示的运行结果。

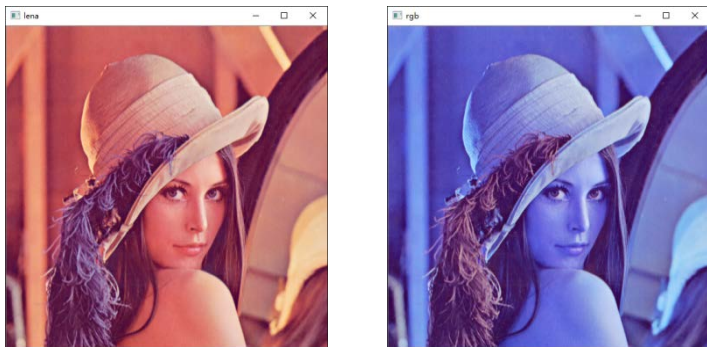


图 4-3 【例 4.5】程序的运行结果

在图 4-3 中，左图是 BGR 通道顺序的图像，右图是 RGB 通道顺序的图像。可以看到，读取的 lena 图像在 BGR 模式下正常显示，将其调整为 RGB 通道顺序后，显示的图像呈现浅蓝色。由于本书为黑白印刷，所以为了更好地观察运行效果，请大家亲自上机运行程序。

4.4 HSV 色彩空间讨论

RGB 色彩空间是一种被广泛接受的色彩空间，但是该色彩空间过于抽象，我们不能够直接通过其值感知具体的色彩。我们更习惯使用直观的方式来感知颜色，HSV 色彩空间提供了这样的方式。通过 HSV 色彩空间，我们能够更加方便地通过色调、饱和度和亮度来感知颜色。

其实，除了 HSV 色彩空间，我们讨论的其他大多数色彩空间都不方便人们对颜色进行理解和解释。例如，现实中我们根本不可能用每种颜料的百分比（RGB 色彩空间）来形容一件衣服的颜色。

4.4.1 基础知识

4.1.4 节已经对 HSV 色彩空间进行了简单介绍，为了方便读者更好地理解后续例题，本节将从其值范围的角度对相关知识点分析说明。



HSV 色彩空间从心理学和视觉的角度出发，提出人眼的色彩知觉主要包含三要素：

- H：色调（Hue，也称为色相）。
- S：饱和度（Saturation）。
- V：亮度（Value）。

1. 色调H

在 HSV 色彩空间中，色调 H 的取值范围是[0,360]。8 位图像内每个像素点所能表示的灰度级有 $2^8=256$ 个，所以在 8 位图像内表示 HSV 图像时，要把色调的角度值映射到[0,255]范围内。在 OpenCV 中，可以直接把色调的角度值除以 2，得到[0,180]之间的值，以适应 8 位二进制（256 个灰度级）的存储和表示范围。

在 HSV 空间中，色调值为 0 表示红色，色调值为 300 表示品红色，具体如表 4-3 所示。

表 4-3 色调值及对应颜色

色调值（度）	颜色
0	红色
60	黄色
120	绿色
180	青色
240	蓝色
300	品红色

根据上述分析可知，每个色调值对应一个指定的色彩，而与饱和度和亮度无关。在 OpenCV 中，将色调值除以 2 之后，会得到如表 4-4 所示的色调值与对应的颜色。

表 4-4 映射后色调值及对应颜色

色调值（度）	颜色
0	红色
30	黄色
60	绿色
90	青色
120	蓝色
150	品红色

确定值范围后，就可以直接在图像的 H 通道内查找对应的值，从而找到特定的颜色。例如，在 HSV 图像中，H 通道内值为 120 的像素点对应蓝色。查找 H 通道内值为 120 的像素点，找到的就是蓝色像素点。

在上述基础上，通过分析各种不同对象对应的 HSV 值，便可以查找不同的对象。例如，通过分析得到肤色的 HSV 值，就可以直接在图像内根据肤色的 HSV 值来查找人脸（等皮肤）区域。

2. 饱和度S

通过 4.1.4 节中的介绍可知，饱和度值的范围是[0,1]，所以针对饱和度，需要说明以下问题：

- 灰度颜色所包含的 R、G、B 的成分是相等的，相当于一种极不饱和的颜色。所以，灰

度颜色的饱和度值是 0。

- 作为灰度图像显示时，较亮区域对应的颜色具有较高的饱和度。
- 如果颜色的饱和度很低，那么它计算所得色调就不可靠。

我们在 4.3 节介绍 `cv2.cvtColor()` 函数时曾指出，进行色彩空间转换后，为了适应 8 位图的 256 个像素级，需要将新色彩空间内的数值映射到 $[0,255]$ 范围内。所以，同样要将饱和度 S 的值从 $[0,1]$ 范围映射到 $[0,255]$ 范围内。

3. 亮度V

通过 4.1.4 节的介绍可知，亮度的范围与饱和度的范围一致，都是 $[0,1]$ 。同样，亮度值在 OpenCV 内也将值映射到 $[0,255]$ 范围内。

亮度值越大，图像越亮；亮度值越低，图像越暗。当亮度值为 0 时，图像是纯黑色。

4.4.2 获取指定颜色

可以通过多种方式获取 RGB 色彩空间的色值在 HSV 色彩空间内所对应的值。例如，可以通过图像编辑软件或者在线网站获取 RGB 值所对应的 HSV 值。

需要注意，在从 RGB/BGR 色彩空间转换到 HSV 色彩空间时，OpenCV 为了满足 8 位图的要求，对 HSV 空间的值进行了映射处理。所以，通过软件或者网站获取的 HSV 值还需要被进一步映射，才能与 OpenCV 中的 HSV 值一致。

在本节中，我们通过程序查看一幅图像在 OpenCV 内从 RGB 色彩空间变换到 HSV 色彩空间后，各个分量的值。

【例 4.6】在 OpenCV 中，测试 RGB 色彩空间中不同颜色的值转换到 HSV 色彩空间后的对应值。

为了方便理解，这里分别对蓝色、绿色、红色的单个像素点进行测试。

首先，使用 `np.zeros([1,1,3],dtype=np.uint8)` 来生成一幅仅有一个像素点的图像（数组）。

接下来，通过对其中的通道分量赋值，将其设定为指定的颜色。例如：

- 语句 `imgBlue[0,0,0]=255` 可以将该像素点的第 0 个通道（即 B 通道）的值设置为 255，即将该点的颜色指定为蓝色。
- 语句 `imgGreen[0,0,1]=255` 可以将该像素点的第 1 个通道（即 G 通道）的值设置为 255，即将该点的颜色指定为绿色。
- 语句 `imgRed[0,0,2]=255` 可以将该像素点的第 2 个通道（即 R 通道）的值设置为 255，即将该点的颜色指定为红色。

然后，可以通过语句 `cv2.cvtColor(Blue,cv2.COLOR_BGR2HSV)` 将 Blue 从 BGR 色彩空间转换到 HSV 色彩空间。

最后，通过打印 HSV 色彩空间内的像素值，观察转换情况。

在本例中，对蓝色、绿色、红色三种不同的颜色分别进行转换，将它们从 BGR 色彩空间转换到 HSV 色彩空间，并观察转换后所得到的 HSV 空间的对应值。



根据题目要求及上述分析，设计程序如下：

```
import cv2
import numpy as np
#=====测试一下 OpenCV 中蓝色的 HSV 模式值=====
imgBlue=np.zeros([1,1,3],dtype=np.uint8)
imgBlue[0,0,0]=255
Blue=imgBlue
BlueHSV=cv2.cvtColor(Blue,cv2.COLOR_BGR2HSV)
print("Blue=\n",Blue)
print("BlueHSV=\n",BlueHSV)
#=====测试一下 OpenCV 中绿色的 HSV 模式值=====
imgGreen=np.zeros([1,1,3],dtype=np.uint8)
imgGreen[0,0,1]=255
Green=imgGreen
GreenHSV=cv2.cvtColor(Green,cv2.COLOR_BGR2HSV)
print("Green=\n",Green)
print("GreenHSV=\n",GreenHSV)
#=====测试一下 OpenCV 中红色的 HSV 模式值=====
imgRed=np.zeros([1,1,3],dtype=np.uint8)
imgRed[0,0,2]=255
Red=imgRed
RedHSV=cv2.cvtColor(Red,cv2.COLOR_BGR2HSV)
print("Red=\n",Red)
print("RedHSV=\n",RedHSV)
```

运行程序，会显示如下所示的运行结果：

```
Blue=
[[[255  0  0]]]
BlueHSV=
[[[120 255 255]]]
Green=
[[[ 0 255  0]]]
GreenHSV=
[[[ 60 255 255]]]
Red=
[[[ 0  0 255]]]
RedHSV=
[[[ 0 255 255]]]
```

从运行结果可以看到，各种颜色的值与表 4-4 所列出的情况一致。

4.4.3 标记指定颜色

在 HSV 色彩空间中，H 通道（饱和度 Hue 通道）对应不同的颜色。或者换个角度理解，颜色的差异主要体现在 H 通道值的不同上。所以，通过对 H 通道值进行筛选，便能够筛选出特定的颜色。例如，在一幅 HSV 图像中，如果通过控制仅仅将 H 通道内值为 240（在 OpenCV

内被调整为 120) 的像素显示出来, 那么图像中就会仅仅显示蓝色部分。

本节将首先通过例题展示一些实现上的细节问题, 然后通过具体例题展示如何将图像内的特定颜色标记出来, 即将一幅图像内的其他颜色屏蔽, 仅仅将特定颜色显示出来。

1. 通过inRange函数锁定特定值

OpenCV 中通过函数 `cv2.inRange()` 来判断图像内像素点的像素值是否在指定的范围内, 其语法格式为:

```
dst = cv2.inRange( src, lowerb, upperb )
```

式中:

- `dst` 表示输出结果, 大小和 `src` 一致。
- `src` 表示要检查的数组或图像。
- `lowerb` 表示范围下界。
- `upperb` 表示范围上界。

返回值 `dst` 与 `src` 等大小, 其值取决于 `src` 中对应位置上的值是否处于区间`[lowerb,upperb]`内:

- 如果 `src` 值处于该指定区间内, 则 `dst` 中对应位置上的值为 255。
- 如果 `src` 值不处于该指定区间内, 则 `dst` 中对应位置上的值为 0。

【例 4.7】 使用函数 `cv2.inRange()` 将某个图像内的在`[100,200]`内的值标注出来。

为了方便理解, 这里采用一个二维数组模拟图像, 完成操作。

根据题目要求, 设计程序如下:

```
import cv2
import numpy as np
img=np.random.randint(0,256,size=[5,5],dtype=np.uint8)
min=100
max=200
mask = cv2.inRange(img, min, max)
print("img=\n",img)
print("mask=\n",mask)
```

运行程序, 会显示如下所示的运行结果。

```
img=
[[129 155 99 51 182]
 [ 57 130 235 135 110]
 [232 182 194 13 26]
 [111 7 136 190 55]
 [ 35 144 9 255 187]]
mask=
[[255 255 0 0 255]
 [ 0 255 0 255 255]
 [ 0 255 255 0 0]
 [255 0 255 255 0]]
```



```
[ 0 255  0  0 255]]
```

通过本例题可以看出，通过函数 `cv2.inRange()` 可以将数组（图像）内指定范围内的值标注出来，在返回的 `mask` 中，其值取决于 `img` 中对应位置上的值是否在 `inRange` 所指定的 `[100,200]` 内：

- 如果 `img` 值位于该指定区间内，则 `mask` 对应位置上的值为 255。
- 如果 `img` 值不在该指定区间内，则 `mask` 对应位置上的值为 0。

返回的结果 `mask` 可以理解为一个掩码数组，其大小与原始数组一致。

2. 通过基于掩码的按位与显示ROI

【例 4.8】 正常显示某个图像内的感兴趣区域（ROI），而将其余区域显示为黑色。

为了方便理解，这里采用一个二维数组模拟图像，完成操作。题目中要求将不感兴趣区域以黑色显示，可以通过设置掩码的方式将该区域的值置为 0 来实现。

根据题目要求，设计程序如下：

```
import cv2
import numpy as np
img=np.ones([5,5],dtype=np.uint8)*9
mask =np.zeros([5,5],dtype=np.uint8)
mask[0:3,0]=1
mask[2:5,2:4]=1
roi=cv2.bitwise_and(img,img, mask= mask)
print("img=\n",img)
print("mask=\n",mask)
print("roi=\n",roi)
```

在本例中，通过 `mask` 设置了两个感兴趣区域（掩模）。后续通过在按位与运算中设置掩模的方式，将原始图像 `img` 内这两部分的值保留显示，而将其余部分的值置零。

运行程序，会显示如下所示的运行结果：

```
img=
[[9 9 9 9 9]
 [9 9 9 9 9]
 [9 9 9 9 9]
 [9 9 9 9 9]
 [9 9 9 9 9]]
mask=
[[1 0 0 0 0]
 [1 0 0 0 0]
 [1 0 1 1 0]
 [0 0 1 1 0]
 [0 0 1 1 0]]
roi=
[[9 0 0 0 0]
 [9 0 0 0 0]
 [9 0 9 9 0]
 [0 0 9 9 0]
 [0 0 9 9 0]]
```

```
[0 0 9 9 0]
[0 0 9 9 0]]
```

3. 显示特定颜色值

【例 4.9】 分别提取 OpenCV 的 logo 图像内的红色、绿色、蓝色。

需要注意，在实际提取颜色时，往往不是提取一个特定的值，而是提取一个颜色区间。例如，在 OpenCV 中的 HSV 模式内，蓝色在 H 通道内的值是 120。在提取蓝色时，通常将“蓝色值 120”附近的一个区间的值作为提取范围。该区间的半径通常为 10 左右，例如通常提取 [120-10,120+10]范围内的值来指定蓝色。

相比之下，HSV 模式中 S 通道、V 通道的值的取值范围一般是[100,255]。这主要是因为，当饱和度和亮度太低时，计算出来的色调可能就不可靠了。

根据上述分析，各种颜色的 HSV 区间值分布在[H-10,100,100]和[H+10,255,255]之间。因此，各种颜色值的范围为：

- 蓝色：值分布在[110,100,100]和[130,255,255]之间。
- 绿色：值分布在[50,100,100]和[70,255,255]之间。
- 红色：值分布在[0,100,100]和[10,255,255]之间。

根据前述例题的相关介绍，首先利用函数 cv2.inRange()查找指定颜色区域，然后利用基于掩码的按位与运算将指定颜色提取出来。

根据题目要求，设计程序如下：

```
import cv2
import numpy as np
opencv=cv2.imread("opencv.jpg")
hsv = cv2.cvtColor(opencv, cv2.COLOR_BGR2HSV)
cv2.imshow('opencv',opencv)
#=====指定蓝色值的范围=====
minBlue = np.array([110,50,50])
maxBlue = np.array([130,255,255])
#确定蓝色区域
mask = cv2.inRange(hsv, minBlue, maxBlue)
#通过掩码控制的按位与运算，锁定蓝色区域
blue = cv2.bitwise_and(opencv,opencv, mask= mask)
cv2.imshow('blue',blue)
#=====指定绿色值的范围=====
minGreen = np.array([50,50,50])
maxGreen = np.array([70,255,255])
#确定绿色区域
mask = cv2.inRange(hsv, minGreen, maxGreen)
#通过掩码控制的按位与运算，锁定绿色区域
green = cv2.bitwise_and(opencv,opencv, mask= mask)
cv2.imshow('green',green)
#=====指定红色值的范围=====
minRed = np.array([0,50,50])
```




```

maxRed = np.array([30,255,255])
#确定红色区域
mask = cv2.inRange(hsv, minRed, maxRed)
#通过掩码控制的按位与运算，锁定红色区域
red= cv2.bitwise_and(opencv,opencv, mask= mask)
cv2.imshow('red',red)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行程序，结果如图 4-4 所示，其中：

- 图(a)是原始图像。
- 图(b)是从图(a)中提取得到的蓝色部分。
- 图(c)是从图(a)中提取得到的绿色部分。
- 图(d)是从图(a)中提取得到的红色部分。

由于本书为黑白印刷，所以为了更好地观察运行效果，请大家亲自上机运行程序。

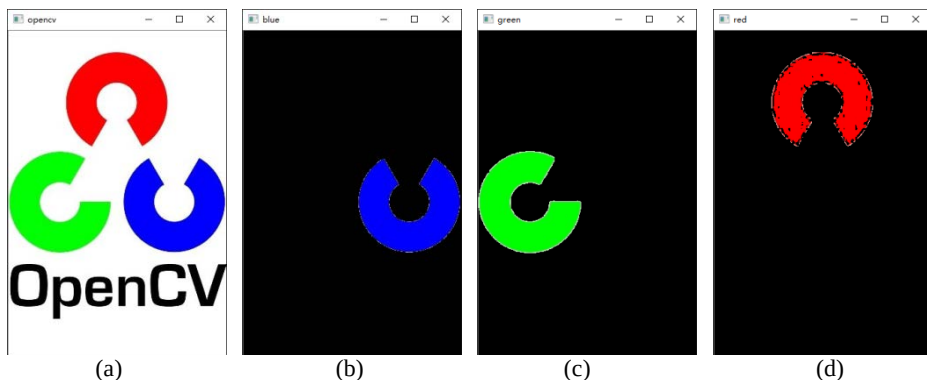


图 4-4 【例 4.9】程序的运行结果

4.4.4 标记肤色

在标记特定颜色的基础上，可以将标注范围进一步推广到特定的对象上。例如，通过分析可以估算出肤色在 HSV 色彩空间内的范围值。在 HSV 空间内筛选出肤色范围内的值，即可将图像内包含肤色的部分提取出来。

这里将肤色范围划定为：

- 色调值在[5, 170]之间
- 饱和度值在[25, 166]之间

【例 4.10】提取一幅图像内的肤色部分。

根据题目要求，设计程序如下：

```

import cv2
img=cv2.imread("lesson2.jpg")
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
h,s,v=cv2.split(hsv)

```



```

minHue=5
maxHue=170
hueMask=cv2.inRange(h, minHue, maxHue)
minSat=25
maxSat=166
satMask = cv2.inRange(s, minSat, maxSat)
mask = hueMask & satMask
roi = cv2.bitwise_and(img,img, mask= mask)
cv2.imshow("img",img)
cv2.imshow("ROI",roi)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行程序，结果如图 4-5 所示，程序实现了将人的图像从背景内分离出来。其中：

- 左侧是原始图像，图像背景是白色的。
- 右侧是提取结果，提取后的图像仅保留了人像肤色（包含衣服）部分，背景为黑色。

由于本书是黑白印刷，无法很好地感受到颜色的变化。为了更好地观察上述效果，请大家亲自上机运行程序。

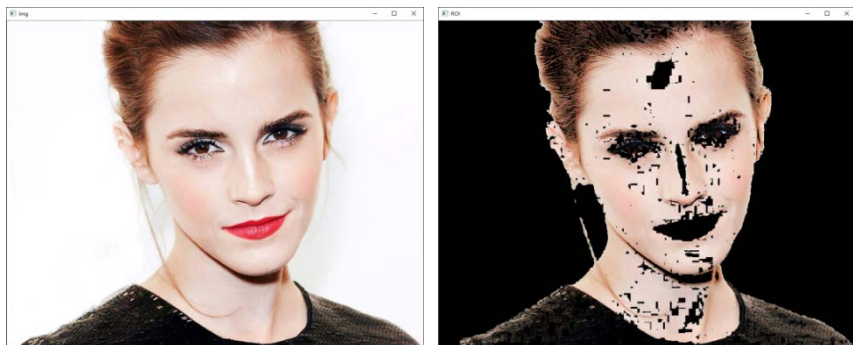


图 4-5 【例 4.10】程序的运行结果

4.4.5 实现艺术效果

在 HSV 色彩空间内进行分量值的调整能够生成一些有趣的效果。一些图像处理软件正是利用对 HSV 色彩空间内值的调整来实现各种艺术效果的。

在本节中，将一幅图像的 H 通道和 S 通道的值保持不变，而将其 V 通道的值都调整为 255，即设置为最亮，观察得到的艺术效果。

【例 4.11】调整 HSV 色彩空间内 V 通道的值，观察其处理结果。

可以任意改变图像内各个通道的值，观察其最终的显示效果。本例中，我们改变 V 通道的值，让其值均变为 255，观察图像处理结果。

根据题目要求，设计程序如下：

```

import cv2
img=cv2.imread("barbara.bmp")
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

```



```

h, s, v=cv2.split(hsv)
v[:, :]=255
newHSV=cv2.merge([h, s, v])
art = cv2.cvtColor(newHSV, cv2.COLOR_HSV2BGR)
cv2.imshow("img",img)
cv2.imshow("art",art)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行程序，结果如图 4-6 所示。其中，左侧是原始图像，右侧是艺术效果。

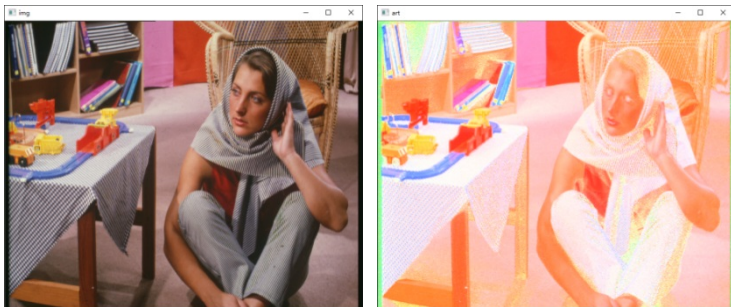


图 4-6 艺术效果

4.5 alpha 通道

在 RGB 色彩空间三个通道的基础上，还可以加上一个 A 通道，也叫 alpha 通道，表示透明度。这种 4 个通道的色彩空间被称为 RGBA 色彩空间，PNG 图像是一种典型的 4 通道图像。alpha 通道的赋值范围是[0, 1]，或者[0, 255]，表示从透明到不透明。

【例 4.12】编写一个程序，分析 alpha 通道的值。

为了方便观察，本例中使用一个随机数组来模拟图像，进行观察。

根据题目要求，设计程序如下：

```

import cv2
import numpy as np
img=np.random.randint(0,256,size=[2,3,3],dtype=np.uint8)
bgra = cv2.cvtColor(img, cv2.COLOR_BGR2BGRA)
print("img=\n",img)
print("bgra=\n",bgra)
b,g,r,a=cv2.split(bgra)
print("a=\n",a)
a[:, :]=125
bgra=cv2.merge([b,g,r,a])
print("bgra=\n",bgra)

```

在本例中，使用语句 `bgra = cv2.cvtColor(img, cv2.COLOR_BGR2BGRA)` 将 `img` 从 BGR 色彩空间转换到 BGRA 色彩空间。在转换后的 BGRA 色彩空间中，A 是 alpha 通道，默认值为 255。

接下来，分别使用打印语句打印原始图像 `img` 的值和转换后的图像 `bgra` 的值。

然后，使用语句 `a[:, :]=125` 将从 `bgra` 中提取的 `alpha` 通道的值设定为 125，并使用语句 `bgra=cv2.merge([b,g,r,a])` 构建一个新的 `bgra` 图像。在本步骤中，使用 `cv2.merge()` 函数将新的 `alpha` 通道与原有的 `BGR` 通道进行合并，得到一个新的图像。从另外一个角度理解就是，本步骤实现了将 `bgra` 图像中 `alpha` 通道的值更改为 125。

最后，使用 `print` 语句显示重构后的 `bgra` 图像。

运行程序，结果如下所示。

```
img=
[[[141  62  75]
  [ 64  55 238]
  [ 10 167 220]]

 [[ 19  29  93]
  [234 219 238]
  [108  33  99]]]
bgra=
[[[141  62  75 255]
  [ 64  55 238 255]
  [ 10 167 220 255]]

 [[ 19  29  93 255]
  [234 219 238 255]
  [108  33  99 255]]]
a=
[[255 255 255]
 [255 255 255]]
bgra=
[[[141  62  75 125]
  [ 64  55 238 125]
  [ 10 167 220 125]]

 [[ 19  29  93 125]
  [234 219 238 125]
  [108  33  99 125]]]
```

【例 4.13】 编写一个程序，对图像的 `alpha` 通道进行处理。

根据题目要求，设计程序如下：

```
import cv2
img=cv2.imread("lenacolor.png")
bgra = cv2.cvtColor(img, cv2.COLOR_BGR2BGRA)
b,g,r,a=cv2.split(bgra)
a[:, :]=125
bgra125=cv2.merge([b,g,r,a])
a[:, :]=0
```



```

bgra0=cv2.merge([b,g,r,a])
cv2.imshow("img",img)
cv2.imshow("bgra",bgra)
cv2.imshow("bgra125",bgra125)
cv2.imshow("bgra0",bgra0)
cv2.waitKey()
cv2.destroyAllWindows()
cv2.imwrite("bgra.png", bgra)
cv2.imwrite("bgra125.png", bgra125)
cv2.imwrite("bgra0.png", bgra0)

```

在本例中，首先从当前目录下读取文件 `lenacolor.png`，然后将其进行色彩空间变换，将其由 BGR 色彩空间转换到 BGRA 色彩空间，得到 `bgra`，即为原始图像 `lena` 添加 `alpha` 通道。

接下来，分别将提取得到的 `alpha` 通道的值设置为 125、0，并将新的 `alpha` 通道与原有的 BGR 通道进行组合，得到新的 BGRA 图像 `bgra125`、`bgra0`。

接着，分别显示原始图像、原始 BGRA 图像 `bgra`、重构的 BGRA 图像 `bgra125` 和 `bgra0`。最后，将 3 个不同的 BGRA 图像保存在当前目录下。

运行程序，显示的图像如图 4-7 所示。图中：

- 图(a)是原始图像 `lena`。
- 图(b)是由原始图像 `lena` 通过色彩空间转换得到的图像 `bgra`，该图像内 `alpha` 通道的值是默认值 255。
- 图(c)是将图像 `bgra` 中 `alpha` 通道值设置为 0 得到的。
- 图(d)是将图像 `bgra` 中 `alpha` 通道值设置为 125 得到的。

从图中可以看到，各个图像的 `alpha` 通道值虽然不同，但是在显示时是没有差别的。

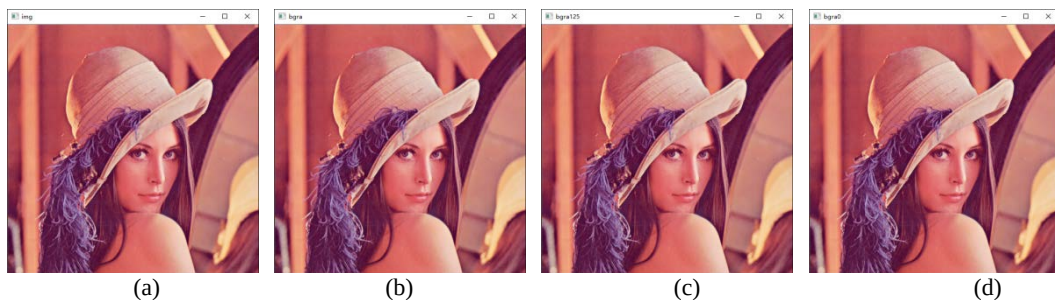


图 4-7 【例 4.13】程序的运行结果

除此以外，程序还分别保存了不同 `alpha` 通道值的图像。打开当前文件夹，可以看到当前文件夹下保存了三幅图像，如图 4-8 所示，其中：

- 图(a)是保存的图像 `bgra`，该图像由原始图像 `lena` 通过色彩空间转换得到，该图像内 `alpha` 通道的值是默认值 255。
- 图(b)是保存的图像 `bgra125`，该图像是将图像 `bgra` 中 `alpha` 通道值设置为 125 得到的。
- 图(c)是保存的图像 `bgra0`，该图像是将图像 `bgra` 中 `alpha` 通道值设置为 0 得到的。需要注意，在图像 `bgra0` 处于预览模式时，看起来可能是一幅黑色的图像，将其打开后就会

看到它实际上是纯色透明的。



图 4-8 保存的图像